Vault with Kerberos

What is Vault?

Vault is an identity-based secrets and encryption management system. A secret is anything that you want to tightly control access to, such as API encryption keys, passwords, or certificates. Vault provides encryption services that are gated by authentication and authorization methods. Using Vault's UI, CLI, or HTTP API, access to secrets and other sensitive data can be securely stored and managed, tightly controlled (restricted), and auditable.

Examples of Vault use cases:

- Automated PKI Infrastructure
- Data Encryption & Tokenization
- Database Credential Rotation
- Dynamic Secrets
- Identity-based Access
- Key Management
- Kubernetes Secrets
- Secrets Management

Why Vault is especially significant in tandem with Kerberos

CERN is a bit different in one crucial way: there is extra trust between the inventory (inventory management system, network DB, DNS) and the Kerberos authentication.

If provisioning takes care of both building a machine and adding it to inventory, then provided that the provisioning process and inventory can be trusted you can add a link between inventory and Kerberos auth.

Machines can (and already do!) ask for authentication with Kerberos automatically (on first boot for example), then Kerberos itself verifies machine credentials with inventory to authenticate it.

This needs pretty tight ACLs and very good trust in security and administration of inventory and Kerberos, but done correctly solves the zero secret problem - no more need to manually insert secrets.

On successful authentication the machine can request internally trusted self-signed certificates from the CA. If we already trust this process then asking for stored (Hashicorp Vault) or encrypted (SOPS, Ansible Vault) configuration secrets is a possibility as well. This opens a whole can of security worms, *but* a whole world of possibilites too!

How do I access Vault?

Use the CLI tool, by installing it and setting your environment variable `VAULT_ADDR='https://vault.cern.ch:8200'`

OR

Use the web UI by navigating to https://vault.cern.ch:8200 in your browser and log in using LDAP method by using your CERN account credentials.

ADMIN GUIDE

Following sections describes what the Vault back-end admin must do in order to enable LDAP and Kerberos integrations to work with CERN auth methods and also how to map LDAP groups to Vault policies.

Necessary first step

Current CERN Kerberos configuration includes the lines which are incompatible with Vault as parsing these lines leads Vault to believe that the Kerberos is using v4 instead of v5:

Text Only

```
v4_name_convert = {
  host = {
    rcmd = host
  }
}
```

Duplicate Kerberos configuration:

Bash

```bash
cp /etc/krb5.conf /etc/krb5.conf.nov4
```

Remove incompatible Kerberos v4 lines (the following command finds the line `v4_name_convert` and deletes it + 4 lines after it in a file):

Bash

```bash
sed -i "$(grep -n v4_name_convert /etc/krb5.conf.nov4 | cut -d : -f1),$(expr $(grep -n v4_name_convert /etc/krb5.conf.nov4 | cut -d : -f1) + 4)d" /etc/krb5.conf.nov4
```

LDAP config (user auth)

Official Vault LDAP documentation and API documentation.

Enable the LDAP auth method:

Bash

```bash
vault auth enable ldap
```

Configure the LDAP auth method (for user auth):

Bash

```bash


```

```
vault write auth/ldap/config \
  binddn="CN=<SERVICE_USER>,OU=Users,OU=Organic Units,DC=cern,dc=ch" \
  bindpass="<SERVICE_PASSWORD>" \
  url="ldap://cerndc.cern.ch" \
  userattr=sAMAccountName \
  userdn="DC=cern,DC=ch" \
  groupfilter="(&(objectClass=group)(member:1.2.840.113556.1.4.1941:=
{{.UserDN}}))" \
  groupattr=memberOf \
  groupdn="OU=Workgroups,DC=cern,DC=ch" \
  certificate="@/etc/openldap/cacerts/CERN_Root_Certification_Authority_2.pem"
\
  insecure_tls=false \
  starttls=true \
  discoverdn=false \
  case_sensitive_names=true \
  use_token_groups=true \
  upndomain="" \
  username_as_alias=true
```

Arguments in detail:

- binddn - Distinguished name of object to bind when performing user and group search. Example: `cn=vault,ou=Users,dc=example,dc=com`. Used together with `bindpass` argument.
- bindpass - Password to use along with `binddn` argument when performing user search.
- url - The LDAP server to connect to. Examples: ldaps://xldap.cern.ch, ldap://cerndc.cern.ch, ldap://tndc.cern.ch (Check HERE for more details). This can also be a comma-delineated list of URLs, in which case the servers will be tried in-order if there are errors during the connection process.
- userattr - Attribute on user attribute object matching the username passed when authenticating. Examples: `sAMAccountName`, `cn`, `uid`.
- userdn - Base DN under which to perform user search. Example: `ou=Users,dc=example,dc=com`.
- groupfilter - Go template used when constructing the group membership query. The template can access the following context variables: [`UserDN`, `Username`]. The default is `(|(memberUid={{.Username}})(member={{.UserDN}})(uniqueMember={{.UserDN}}))`, which is compatible with several common directory schemas. To support nested group resolution for Active Directory, instead use the following query: `(&(objectClass=group)(member:1.2.840.113556.1.4.1941:={{.UserDN}}))`. Both examples work with CERN LDAP schema.

- groupattr - LDAP attribute to follow on objects returned by `groupfilter` in order to enumerate user group membership. Examples: for groupfilter queries returning *group* objects, use: `cn`. For queries returning *user* objects, use: `memberOf`. The default is `cn`.
- groupdn - LDAP search base to use for group membership search. This can be the root containing either groups or users. Example: `ou=Groups,dc=example,dc=com`
- certificate - CA certificate to use when verifying LDAP server certificate, must be x509 PEM encoded. CERN certs are under `/etc/openldap/cacerts`
- insecure_tls - If true, skips LDAP server SSL certificate verification - insecure, use with caution!
- starttls - If true, issues a `StartTLS` command after establishing an unencrypted connection.
- discoverdn - If true, use anonymous bind to discover the bind DN of a user.
- case_sensitive_names - If set, user and group names assigned to policies within the backend will be case sensitive. Otherwise, names will be normalized to lower case. Case will still be preserved when sending the username to the LDAP server at login time; this is only for matching local user/group definitions.
- use_token_groups - If true, groups are resolved through Active Directory tokens. This speeds up nested group membership resolution in large directories significantly. Highly recommended to enable this with CERN LDAP.
- upndomain - userPrincipalDomain used to construct the UPN string for the authenticating user. The constructed UPN will appear as `[username]@UPNDomain`. Example: `[example.com](http://example.com)`, which will cause vault to bind as `[username@example.com](mailto:username@example.com)`. CERN uses auth by username not by email so this must not be set.
- username_as_alias - If set to true, forces the auth method to use the username passed by the user as the alias name. (This would be convenient, but seems to not work for host keytab logins)

Map Vault policies to LDAP groups:

Bash

```bash
vault write auth/ldap/groups/<LDAP GROUP NAME> policies=<VAULT POLICY NAME>
```

Kerberos config (host auth)

Official Vault Kerberos documentation and API documentation.

Enable Kerberos authentication in Vault:

Bash

```bash
vault auth enable \
  -passthrough-request-headers=Authorization \
  -allowed-response-headers=www-authenticate \
  kerberos
```

Encode keytab for the Kerberos plugin (on the same machine Vault service is deployed on):

Bash

```bash
base64 /etc/krb5.keytab > krb5.keytab.base64
```

Configure the Kerberos auth method with the encoded keytab and entry name that will be used to verify inbound login requests:

Bash

```bash
vault write auth/kerberos/config \
  keytab=@krb5.keytab.base64 \
  service_account="host/vault.cern.ch"
```

NOTE: `service_account` must match a valid principal in the keytab!

Configure the Kerberos auth method to communicate with LDAP (for host secrets):

Bash

```bash
```

```
vault write auth/kerberos/config/ldap \
  binddn="CN=<SERVICE_USER>,OU=Users,OU=Organic Units,DC=cern,dc=ch" \
  bindpass="<SERVICE_PASSWORD>" \
  url="ldap://cerndc.cern.ch" \
  userattr=sAMAccountName \
  userdn="DC=cern,DC=ch" \
  groupfilter="(&(objectClass=group)(member:1.2.840.113556.1.4.1941:=
{{.UserDN}}))" \
  groupattr=memberOf \
  groupdn="OU=Workgroups,DC=cern,DC=ch" \
  certificate="@/etc/openldap/cacerts/CERN_Root_Certification_Authority_2.pem"
\
  insecure_tls=false \
  starttls=true \
  discoverdn=false \
  case_sensitive_names=true \
  use_token_groups=true \
  upndomain="" \
  username_as_alias=true
```

Arguments are described in detail in the LDAP configuration section and not duplicated here as they are identical.

Map Vault policies to LDAP groups:

Bash

```
vault write auth/kerberos/groups/<LDAP GROUP NAME> policies=<VAULT POLICY
NAME>
```

Creating policies

Official Vault Policies documentation

Everything in Vault is path based, and policies are no exception. Policies provide a declarative way to grant or forbid access to certain paths and operations in Vault. This section discusses policy workflows and syntaxes.

Policies are **deny by default**, so an empty policy grants no permission in the system.

Syntax and usage

Policies are written in HCL or JSON and describe which paths in Vault a user or machine is allowed to access.

Here is a very simple policy which grants read capabilities to the path `secret/foo`:

Text Only

```
path "secret/foo" {
  capabilities = ["read"]
}
```

When this policy is assigned to a token, the token can read from `secret/foo`. However, the token cannot update or delete `secret/foo`, since the capabilities do not allow it. Because policies are deny by default, the token would have no other access in Vault.

Here is a more detailed policy, and it is documented inline:

Text Only

```
# This section grants all access on "secret/*". Further restrictions can be
# applied to this broad policy, as shown below.
path "secret/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
}

# Even though we allowed secret/*, this line explicitly denies
# secret/super-secret. This takes precedence.
path "secret/super-secret" {
  capabilities = ["deny"]
}

# Policies can also specify allowed, disallowed, and required parameters.
Here
# the key "secret/restricted" can only contain "foo" (any value) and "bar"
(one
# of "zip" or "zap").
path "secret/restricted" {
  capabilities = ["create"]
  allowed_parameters = {
    "foo" = []
    "bar" = ["zip", "zap"]
  }
}
```

Policies use path-based matching to test the set of capabilities against a request. A policy `path` may specify an exact path to match, or it could specify a glob pattern which instructs Vault to use a prefix match:

Text Only

```

```

```
# Permit reading only "secret/foo". An attached token cannot read "secret/food"
# or "secret/foo/bar".
path "secret/foo" {
  capabilities = ["read"]
}

# Permit reading everything under "secret/bar". An attached token could read
# "secret/bar/zip", "secret/bar/zip/zap", but not "secret/bars/zip".
path "secret/bar/*" {
  capabilities = ["read"]
}

# Permit reading everything prefixed with "zip-". An attached token could read
# "secret/zip-zap" or "secret/zip-zap/zong", but not "secret/zip/zap
path "secret/zip-*" {
  capabilities = ["read"]
}
```

In addition, a `+` can be used to denote any number of characters bounded within a single path segment:

Text Only

```
# Permit reading the "teamb" path under any top-level path under secret/
path "secret/+/teamb" {
  capabilities = ["read"]
}

# Permit reading secret/foo/bar/teamb, secret/bar/foo/teamb, etc.
path "secret/+/+/teamb" {
  capabilities = ["read"]
}
```

Path capabilities cheat sheet

`create` (`POST/PUT`) - Allows creating data at the given path. Very few parts of Vault distinguish between `create` and `update`, so most operations require both `create` and `update` capabilities. Parts of Vault that provide such a distinction are noted in documentation.

`read` (`GET`) - Allows reading the data at the given path.

`update` (`POST/PUT`) - Allows changing the data at the given path. In most parts of Vault, this implicitly includes the ability to create the initial value at the path.

`delete` (`DELETE`) - Allows deleting the data at the given path.

`list` (`LIST`) - Allows listing values at the given path. Note that the keys returned by a `list` operation are *not* filtered by policies. Do not encode sensitive information in key names. Not all backends support listing.

In addition to the standard set, there are some capabilities that do not map to HTTP verbs.

`sudo` - Allows access to paths that are *root-protected*. Tokens are not permitted to interact with these paths unless they have the `sudo` capability (in addition to the other necessary capabilities for performing an operation against that path, such as `read` or `delete`).

For example, modifying the audit log backends requires a token with `sudo` privileges.

`deny` - Disallows access. This always takes precedence regardless of any other defined capabilities, including `sudo`.

Templating policies

Policy templating allows the automating mapping of paths to certain users for example. Admin has no way of knowing who exactly will log into Vault and it would be very time inefficient to create a path for each user's personal secrets manually, templating enables to automate this.

Name	Description
`identity.entity.id`	The entity's ID
`identity.entity.name`	The entity's name
`identity.entity.metadata.<metadata key>`	Metadata associated with the entity for the given key
`identity.entity.aliases.<mount accessor>.id`	Entity alias ID for the given mount
`identity.entity.aliases.<mount accessor>.name`	Entity alias name for the given mount

`identity.entity.aliases.<mount accessor>.metadata.<metadata key>`	Metadata associated with the alias for the given mount and metadata key
`identity.entity.aliases.<mount accessor>.custom_metadata.<custom_metadata key>`	Custom metadata associated with the alias for the given mount and custom metadata key
`identity.groups.ids.<group id>.name`	The group name for the given group ID
`identity.groups.names.<group name>.id`	The group ID for the given group name
`identity.groups.ids.<group id>.metadata.<metadata key>`	Metadata associated with the group for the given key
`identity.groups.names.<group name>.metadata.<metadata key>`	Metadata associated with the group for the given key

Examples:

If you wanted to create a shared section of KV that is associated with entities that are in a group:

Text Only

```
# In the example below, the group ID maps a group and the path
path "secret/data/groups/{{identity.groups.ids.fb036ebc-2f62-4124-9503-
42aa7A869741.name}}/*" {
  capabilities = ["create", "update", "read", "delete"]
}

path "secret/metadata/groups/{{identity.groups.ids.fb036ebc-2f62-4124-9503-
42aa7A869741.name}}/*" {
  capabilities = ["list"]
}
```

If you wanted to create personal secrets path for every use that logs in using LDAP:

Text Only

```

```

```
# Anyone who logs in using LDAP with accessor auth_ldap_35641cad can create, update, read, delete secrets under the path kv/personal/<LDAP USERNAME>
path "kv/personal/{{identity.entity.aliases.auth_ldap_35641cad.name}}/*" {
  capabilities = [ "create", "list", "update", "read", "delete" ]
}
```

CLIENT GUIDE

Necessary first step (client)

Current CERN Kerberos configuration includes the lines which are incompatible with Vault as parsing these lines leads Vault to believe that the Kerberos is using v4 instead of v5:

Text Only

```
v4_name_convert = {
  host = {
    rcmd = host
  }
}
```

Duplicate Kerberos configuration:

Bash

```
cp /etc/krb5.conf /etc/krb5.conf.nov4
```

Remove incompatible Kerberos v4 lines (the following command finds the line `v4_name_convert` and deletes it + 4 lines after it in a file):

Bash

```
sed -i "$(grep -n v4_name_convert /etc/krb5.conf.nov4 | cut -d : -f1),$(expr $(grep -n v4_name_convert /etc/krb5.conf.nov4 | cut -d : -f1) + 4)d" /etc/krb5.conf.nov4
```

User auth

The easiest way for users to authenticate to Vault is using the LDAP method.

Authenticate to Vault using Vault CLI:

Bash

```
vault login -method=ldap username=<YOUR USERNAME>
```

Host auth

The easiest way of automating access to Vault for hosts, deployment of apps, services or other autonomous tasks is to leverage keytabs and use the Vault's Kerberos login method. Every CERN host has or can have a valid keytab by using the `cern-get-keytab` CLI tool. The following will leverage this keytab as a means of authenticating to Vault. Please note that you can also create a key for a service as per the official documentation, but this will not provide any meaningful compartmentalization as any automated tasks will still have to be executed as root. The only way to compartmentalize Vault access and secrets is to add hosts to e-groups/LDAP groups and map Vault policies to those groups.

Users can also use the Kerberos methods, but this will require extra steps (like creating a keytab, as the cache can't be used) that may prove this method to be too inconvenient compared to using LDAP.

NOTE: this auth method gets a non-renewable token from Vault! TTL is set by the Vault administrator or whoever manages host auth policies. By default this token will be valid for 768h. Non-renewable token means that the service cannot use the obtained Vault token to request a new Vault token, instead renewal must be done by doing the described process again.

Authenticate to Vault using Vault CLI:

Bash

```
vault login -method=ldap username=<YOUR USERNAME>
```

Arguments in detail:

- username - a valid Kerberos principal in a valid keytab.
- service - a valid principal that Vault back-end should use for obtaining a service ticket for gaining a SPNEGO token. This principal must also exist in LDAP.
- realm - name of the Kerberos realm. This realm must match the UPNDomain configured on the LDAP connection. This check is case-sensitive.
- keytab_path - path to a valid Kerberos ketab. The principal supplied as a username mentioned before must exist in this keytab.
- krb5conf_path - path to a valid Kerberos configuration. Note that some v4 settings will cause Vault to interpret an otherwise valid v5 configuration as invalid.

- disable_fast_negotiation (boolean) - toggle Kerberos auth method's default of using FAST negotiation. FAST is a pre-authentication framework for Kerberos. It includes a mechanism for tunneling pre-authentication exchanges using armoured KDC messages. FAST provides increased resistance to passive password guessing attacks. Some common Kerberos implementations do not support FAST negotiation. CERN Kerberos does not support FAST.

OR authenicate to Vault using REST API:

`/auth/kerberos/login` endpoint allows you to log in with a valid Kerberos SPNEGO token. This token is obtained by the client, marshalled, and converted to base 64 using standard encoding.

TODO: figure out and document a way to obtain a Kerberos SPNEGO token without Vault CLI or other dependencies.

Bash

```bash
curl \
  --header "Authorization: Negotiate <BASE64 SPNEGO TOKEN>" \
  --request POST \
  http://$(VAULT_ADDR):8200/v1/auth/kerberos/login
```

Using Vault

Everything in Vault is path based, you can imagine these paths as similar to a Linux file tree, there is a root and paths spring out from there. Policies (created by admins or moderators of Vault) provide a declarative way to grant or forbid access to certain paths and operations in Vault.

For any of the following to work requires a Vault admin to enable the use of any of the following secret engines.

Key-Value Secrets

Official Vault KV Secrets documentation.

The `kv` secrets engine is a generic Key-Value store used to store arbitrary secrets within the configured physical storage for Vault. This backend can be run in one of two modes; either it can be configured to store a single value for a key or, versioning can be enabled and a configurable number of versions for each key will be stored.

When running the `kv` secrets backend non-versioned, only the most recently written value for a key will be preserved. The benefits of non-versioned `kv` is a reduced storage size for each key since no additional metadata or history is stored. Additionally, requests going to a backend configured this way will be more performant because for any given request there will be fewer storage calls and no locking.

Non-versioned aka KV V1 is easier to set up and requires almost no extra setup.

Secret sharing using key/value V1

Keeping personal secrets, sharing secrets with colleagues can be done most easily by mapping policies to e-groups/LDAP groups and using policy templating. The following chapters documents the usage of using the KV V1 usage and not the KV V2.

SHORT OF IT - CHEAT SHEET

List enabled secrets engines:

Bash

```bash
vault secrets list
```

To set key/value secrets:

- Using CLI: `vault kv put <KV ENGINE NAME>/<PATH> <KEY>=<VALUE>`
- Using API:

Bash

```bash
curl --header "X-Vault-Token: $VAULT_TOKEN" \ --request POST \ --data @payload.json \ $VAULT_ADDR/v1/<KV ENGINE NAME>/<SECRET PATH>
```

Where payload is: `{ "key": "AAaaBBccDDeeOTXzSMT1234BB_Z8JzG7JkSVxI" }`

To get key/value secrets:

- Using CLI:

Bash

```bash
vault kv get <KV ENGINE NAME>/<PATH>
```

- Using API:

Bash

```bash
curl --header "X-Vault-Token: $VAULT_TOKEN" \ $VAULT_ADDR/v1/<KV ENGINE NAME>/<SECRET PATH>
```

If you wish to enter secrets without exposing the secret in my shell's history:

- Using CLI, use a dash "-": `vault kv put <KV ENGINE NAME>/<PATH> <KEY>=-`
- Use dash
- Press enter
- Type the secret
- Press Ctrl+d to end the pipe and write the secret
- Using API, use a JSON file for the payload.

LONG OF IT - A PRACTICAL DEMO

The following policy is something that is set up by the Vault admins, this is not something the client will have to bother with, but is displayed here just to provide extra context. Following example allows anyone with this policy to:

- see all secret *paths* (NOT secret values!) and create new secrets (only if secret does not exist, overwriting is not possible) under the enabled key-value store called `kv` via CLI and web UI.
- create, update, read and delete secrets under the `kv/acc-adm` path.
- create, update, read and delete secrets under their own name path in `kv` personal secrets path aka `kv/personal/<USERNAME>`

Terraform

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
```

```
15161718192021222324
# List all secret paths
# Send write-only secrets to others
path "kv/*" {
  capabilities = [ "create", "list" ]
}

# Web UI usage
path "kv/metadata" {
  capabilities = ["list"]
}

###################### General secrets

# General acc-adm secrets
path "kv/acc-adm/*" {
  capabilities = [ "create", "list", "update", "read",
"delete" ]
}

###################### Personal secrets

# Personal secrets
path
"kv/personal/{{identity.entity.aliases.auth_ldap_35641cad.nar
{
  capabilities = [ "create", "list", "update", "read",
"delete" ]
}
```

Scenario 1 - one way secret sharing with another user

Say that you wanted to share a secret to user called `kturvas`. You would use the following CLI command:

Bash

```
vault kv put kv/personal/kturvas/inbox secret_i_want_to_share=hunter2
```

This command will succeed if `secret_i_want_to_share` does not yet exist, if it exists, you would have to choose another name for the secret, because you do not have the right to update existing secret values.

But you would also notice that you can not read that value you shared with that user (you could test that by trying with `vault kv get kv/personal/kturvas`), because you only have write access.

You would only be able to read, create, update, and delete secrets under your own user path `kv/personal/<YOUR USERNAME>`

So in essence this policy allows anyone to create write-only secrets which makes sharing secrets easy and secure.

Scenario 2 - general secret sharing

Say that you wanted to share a new password for a generic service user `accsvc` that is used by everyone in your group. Let's say that the group is `acc-adm-support` e-group.

Assuming admin linked the policy to the `acc-adm-support` e-group then I would have access to create, update, read and delete secrets under the `kv/acc-adm` path upon login to Vault.

So you would use the following CLI command:

Bash

```bash
vault kv put kv/acc-adm/services/accsvc password=N3wSecur3p4ssw0rD!
```

Transit secrets

The transit secrets engine handles cryptographic functions on data in-transit. Vault doesn't store the data sent to the secrets engine. It can also be viewed as "cryptography as a service" or "encryption as a service". The transit secrets engine can also sign and verify data; generate hashes and HMACs of data; and act as a source of random bytes.

The primary use case for `transit` is to encrypt data from applications while still storing that encrypted data in some primary data store. This relieves the burden of proper encryption/decryption from application developers and pushes the burden onto the operators of Vault.

After the secrets engine is configured and a user/machine has a Vault token with the proper permission, it can use this secrets engine.

USAGE EXAMPLE

Encrypt some plaintext data using the `/encrypt` endpoint with a named key:

NOTE**:** All plaintext data **must be base64-encoded**. The reason for this requirement is that Vault does not require that the plaintext is "text". It could be a binary file such as a PDF or image. The easiest safe transport mechanism for this data as part of a JSON payload is to base64-encode it.

NOTE2: Vault HTTP API imposes a maximum request size of 32MB to prevent a denial of service attack. This can be tuned per `listener` block in the Vault server configuration.

Bash

```bash
vault write transit/encrypt/my-key plaintext=$(base64 <<< "my secret data")
```

You are the retured a key/value combination with the ciphertext.

Decrypt a piece of data using the `/decrypt` endpoint with a named key:

Bash

```bash
vault write transit/decrypt/my-key
ciphertext=vault:v1:8SDd3WHDOjf7mq69CyCqYjBXAiQQAVZRkFM13ok481zoCmHnSeDX9vyf7w=
```

The resulting data is base64-encoded (see the note above for details on why). Decode it to get the raw plaintext:

Bash

```bash
base64 --decode <<< "bXkgc2VjcmV0IGRhdGEK"
```

Transform secrets (requires Vault Enterprise)

The transform secrets engine handles secure data transformation and tokenization against provided input value. The secret engine currently supports `fpe`, `masking`, and `tokenization` as data transformation types. The secret engine currently supports `fpe`, `masking`, and `tokenization` as data transformation types.

USAGE EXAMPLE (TRANSFORM)

After the secrets engine is configured and a user/machine has a Vault token with the proper permission, it can use this secrets engine to encode and decode input values.

Encode some input value using the `/encode` endpoint with a named role:

Bash

```bash
vault write transform/encode/payments value=1111-2222-3333-4444
```

Which returns:

Text Only

```text

```

```
Key              Value
---              -----
encoded_value    9300-3376-4943-8903
```

Decode some input value using the `/decode` endpoint with a named role:

Bash

```bash
vault write transform/decode/payments value=9300-3376-4943-8903
```

Which returns:

Text Only

```
Key              Value
---              -----
decoded_value    1111-2222-3333-4444
```

Decode some input value using the `/decode` endpoint with a named role and decode format:

Bash

```bash
vault write transform/decode/payments/last-four value=9300-3376-4943-8903
```

Which returns:

Text Only

```
Key              Value
---              -----
decoded_value    4444
```

Active Directory secrets

The Active Directory (AD) secrets engine has two main features:

The first feature (password rotation) is where the AD secrets engine rotates AD passwords dynamically. This is designed for a high-load environment where many instances may be accessing a shared password simultaneously. With a simple set up and a simple creds API, it doesn't require instances to be manually registered in advance to gain access. Passwords are lazily rotated based on preset TTLs and can have a length configured to meet your needs. Additionally, passwords can be manually rotated using the rotate-role endpoint.

The second feature (service account check-out) is where a library of service accounts can be checked out by a person or by machines. Vault will automatically rotate the password each time a service account is checked in. Service accounts can be voluntarily checked in, or Vault will check them in when their lending period (or, "ttl", in Vault's language) ends.

OpenLDAP secrets

The OpenLDAP secret engine allows management of LDAP entry passwords as well as dynamic creation of credentials. This engine supports interacting with Active Directory which is compatible with **LDAP v3**.

PKI secrets

The PKI secrets engine generates dynamic X.509 certificates. With this secrets engine, services can get certificates without going through the usual manual process of generating a private key and CSR, submitting to a CA, and waiting for a verification and signing process to complete. Vault's built-in authentication and authorization mechanisms provide the verification functionality.

By keeping TTLs relatively short, revocations are less likely to be needed, keeping CRLs short and helping the secrets engine scale to large workloads. This in turn allows each instance of a running application to have a unique certificate, eliminating sharing and the accompanying pain of revocation and rollover.

In addition, by allowing revocation to mostly be forgone, this secrets engine allows for ephemeral certificates. Certificates can be fetched and stored in memory upon application startup and discarded upon shutdown, without ever being written to disk.

USAGE EXAMPLE (PKI)

Configure a CA certificate and private key. Vault can accept an existing key pair, or it can generate its own self-signed root. In general, we recommend maintaining your root CA outside of Vault and providing Vault a signed intermediate CA:

Bash

```bash
vault write pki/root/generate/internal common_name=my-website.com ttl=8760h
```

Which returns:

Text Only

```
Key              Value
---              -----
certificate      -----BEGIN CERTIFICATE-----...
expiration       1536807433
issuing_ca       -----BEGIN CERTIFICATE-----...
serial_number    7c:f1:fb:2c:6e:4d:99:0e:82:1b:08:0a:81:ed:61:3e:1d:fa:f5:29
```

Update the CRL location and issuing certificates:

Bash

```bash
vault write pki/config/urls issuing_certificates="http://127.0.0.1:8200/v1/pki/ca" crl_distribution_points="http://127.0.0.1:8200/v1/pki/crl"
```

Configure a role that maps a name in Vault to a procedure for generating a certificate. When users or machines generate credentials, they are generated against this role:

Bash

```bash
vault write pki/roles/example-dot-com allowed_domains=my-website.com allow_subdomains=true max_ttl=72h
```

Generate a new credential by writing to the `/issue` endpoint with the name of the role:

Bash

```bash
vault write pki/issue/example-dot-com common_name=www.my-website.com
```

Which returns:

Text Only

```
Key                 Value
---                 -----
certificate         -----BEGIN CERTIFICATE-----...
issuing_ca          -----BEGIN CERTIFICATE-----...
private_key         -----BEGIN RSA PRIVATE KEY-----...
private_key_type    rsa
serial_number
1d:2e:c6:06:45:18:60:0e:23:d6:c5:17:43:c0:fe:46:ed:d1:50:be
```

Database secrets

The database secrets engine generates database credentials dynamically based on configured roles. It works with a number of different databases through a plugin interface.

There are a number of built-in database types, and an exposed framework for running custom database types for extendability. This means that services that need to access a database no longer need to hardcode credentials: they can request them from Vault, and use Vault's leasing mechanism to more easily roll keys. These are referred to as "dynamic roles" or "dynamic secrets".

Since every service is accessing the database with unique credentials, it makes auditing much easier when questionable data access is discovered. You can track it down to the specific instance of a service based on the SQL username.

Vault makes use of its own internal revocation system to ensure that users become invalid within a reasonable time of the lease expiring.

Supported databases:

- Cassandra
- Couchbase
- Elasticsearch
- HanaDB
- IBM Db2
- InfluxDB
- MongoDB
- MongoDB Atlas
- MSSQL
- MySQL/MariaDB
- Oracle
- PostgreSQL
- Redshift
- Snowflake
- Custom

USAGE QUICK GUIDE

CLI

Install Vault according to the official documentation. For CentOS:

Bash

```bash
sudo yum install -y yum-utils
```

Bash

```bash
sudo yum-config-manager --add-repo
https://rpm.releases.hashicorp.com/RHEL/hashicorp.repo
```

Bash

```bash
sudo yum -y install vault
```

CERN AD user usage

NOTE: following requires that you are part of the `acc-adm-support` e-group!

Set Vault address:

Bash

```bash
export VAULT_ADDR='https://vault.cern.ch:8200'
```

Authenticate using LDAP method:

Bash

```bash
vault login -method=ldap username="$USER"
```

List enabled secrets engines:

Bash

```bash
vault secrets list
```

Personal secrets

Personal secrets path is a path that only one account has access to, the path where you create secrets that are specific to your use cases.

Set key/value secrets under your personal user secrets path:

Bash

```
vault kv put kv/personal/"$USER"/demosecret my_first_secret=sUp3rSuD0
```

Note that you are able to create secrets UNDER (`kv/personal/"$USER"/*`) your users path, but no ON (`kv/personal/"$USER"`) it.

Get the secret you just put into Vault:

Bash

```
vault kv get kv/personal/"$USER"/demosecret
```

List all secret paths under your user name:

Bash

```
vault kv list kv/personal/"$USER"/
```

Inbox secrets

Inbox secrets is a path template that allows sharing secrets with one account. Under every personal secret path there is a special sub path called inbox where other users have read/write access to (this is the only path in the personal secrets path where others have read/write access to). That path is `personal/<USERNAME>/inbox/from-<ANY OTHER USERNAME>/*`

Send someone else a secret:

Bash

```
vault kv put kv/personal/kturvas/inbox/from-"$USER"/demosecret
secret_from_"$USER"=HelloVault!
```

Note that you only have access to other people's personal secrets path if it is under the inbox paths `kv/personal/*/inbox/from-"$USER"` and `kv/personal/*/inbox/"$USER"`

General secrets

General secrets correspond to some policy that allows reading/writing secrets shared by a group of users (like users of an e-group/LDAP group). The following requires that you are part of the `acc-adm-support` e-group for example.

Using what you've learned in the previous steps, try creating a secret under the path `kv/acc-adm` and then try fetching it or use the command

```bash
vault kv list
```

Host

Install Vault according to the official documentation. For CentOS:

```bash
sudo yum install -y yum-utils
```

```bash
sudo yum-config-manager --add-repo [https://rpm.releases.hashicorp.com/RHEL/hashicorp.repo](https://rpm.releases.hashicorp.com/RHEL/hashicorp.repo)
```

```bash
sudo yum -y install vault
```

NOTE: this install method is temporary and should be done manually until we decide to have Vault in internal repos. You could also use the rpm on NFS at `/nfs/cs-ccr-nfshome/user/kturvas/starnet/vault-1.10.2-1.x86_64.rpm`

CERN AD host usage

Your host must be part of Computers by OS Unknown group for the following to work since VMs used for testing were only in one group, Computers by OS Unknown, at the time of writing. Of course, the potential of dividing hosts in different e-groups in the future allows for more versatility.

Manual host usage

If you do not wish to add Vault to your host's ansible workflow or would just like to observe how it works in principal.

Create the temporary second Kerberos configuration for Vault using `krb5.conf` as Vault identifies 4 lines of CERN krb5 conf as part of krb4 which it deems incompatible:

Duplicate Kerberos configuration:

Bash

```bash
cp /etc/krb5.conf /etc/krb5.conf.nov4
```

Remove incompatible Kerberos v4 lines (the following command finds the line `v4_name_convert` and deletes it + 4 lines after it in a file):

Bash

```bash
sed -i "$(grep -n v4_name_convert /etc/krb5.conf.nov4 | cut -d : -f1),$(expr $(grep -n v4_name_convert /etc/krb5.conf.nov4 | cut -d : -f1) + 4)d" /etc/krb5.conf.nov4
```

Authenticate to Vault as **root** using the host keytab:

Bash

```bash
vault login -method=kerberos \
  username=host/"$(hostname)" \
  service=host/vault.cern.ch \
  realm=CERN.CH \
  keytab_path=/etc/krb5.keytab \
  krb5conf_path=/etc/krb5.conf.nov4 \
  disable_fast_negotiation=true
```

Now, if your host is also part of Computers by OS Unknown you will be able to **read** (not write or update) secrets under the path `kv/acc-adm/compos`

Authenticate as a user part of `acc-adm-support` to create secrets under that path for hosts to be able to read and fetch them.

Ansible host usage

It is advisable to organise tasks that use Vault in a manner where they are skipped if no connection to Vault is present. This allows using Vault in a very non-intrusive way where all previous workflows will not be affected in case of errors.

Create the temporary second Kerberos configuration for Vault using `krb5.conf` as Vault identifies 4 lines of CERN krb5 conf as part of krb4 which it deems incompatible:

Text Only

```text
```

```
- name: Check if Vault nov4 Kerberos conf exists
  stat:
    path: /etc/krb5.conf.nov4
  register: nov4

- name: Create nov4 Kerberos conf
  shell: |
    cp /etc/krb5.conf /etc/krb5.conf.nov4
    sed -i "$(grep -n v4_name_convert /etc/krb5.conf.nov4 | cut
-d : -f1),$(expr $(grep -n v4_name_convert /etc/krb5.conf.nov4 |
cut -d : -f1) + 4)d" /etc/krb5.conf.nov4
  when: not nov4.stat.exists
```

For example create a task that checks Vault port and registers the result:

Text Only

```
- name: Verify port 8200 is listening on vault.cern.ch
  wait_for:
    host: vault.cern.ch
    port: 8200
    timeout: 1
    register: vault_reachable
  ignore_errors: yes
```

Then add a check to all tasks that use Vault:

Text Only

```
when: not vault_reachable.failed
```

Authenticating and getting a non-renewable token:

Text Only

```
- name: Authenticate to Vault to get a fresh token
  shell: |
    export VAULT_ADDR='{{ vault_addr }}'
    vault login -address="{{ vault_addr }}" -method=kerberos \
    username=host/"$(hostname)" \
    service=host/vault.cern.ch \
    realm=CERN.CH \
    keytab_path=/etc/krb5.keytab \
    krb5conf_path=/etc/krb5.conf.nov4 \
    disable_fast_negotiation=true
  when: not vault_reachable.failed
```

Example of fetching and using secrets (inserting secrets into applications using `sed`):

Text Only

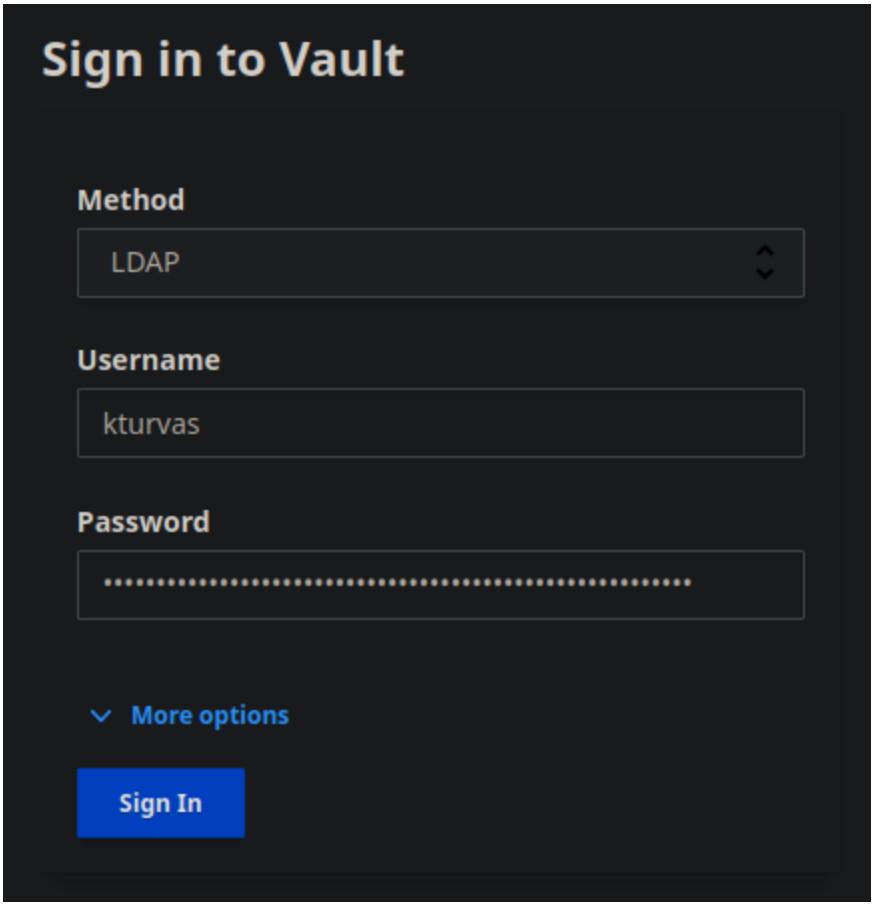
```
- name: Set the broker.json pass
  shell: |
    export BROKER_PASS=$(vault kv get -address='{{ vault_addr }}' -field="{{ fastx_config.broker_pass_var }}" kv/acc-adm/compos/rog)
    sed -i "s/brokerPass/${BROKER_PASS}/" /etc/fastx/broker.json
    sed -i "s/^requirepass.*/requirepass ${BROKER_PASS}/g" /etc/redis/redis.conf
  when: not vault_reachable.failed
```

Web UI

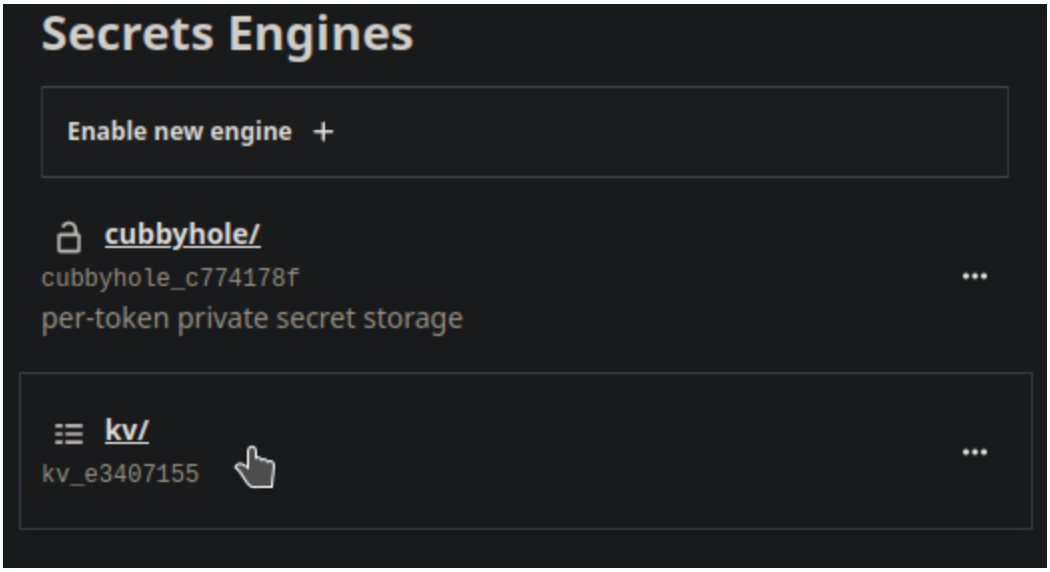
CERN AD user usage

NOTE: following requires that you are part of the acc-adm-support e-group!

Navigate to https://vault.cern.ch:8200 and login using LDAP method with your CERN credentials:



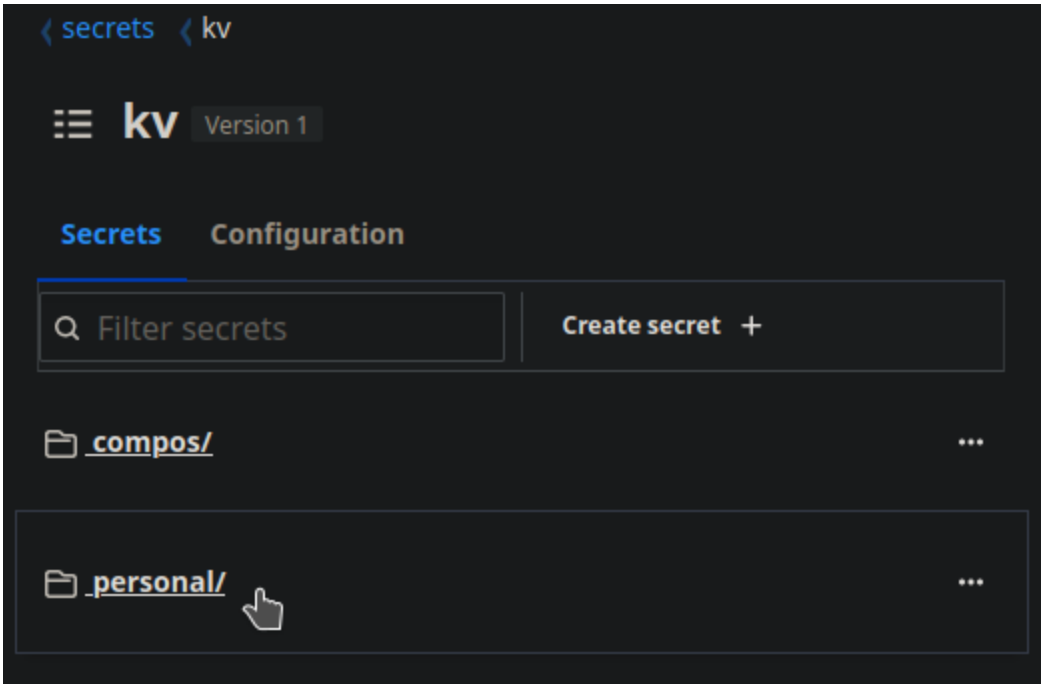
Click on `kv/` secrets engine:



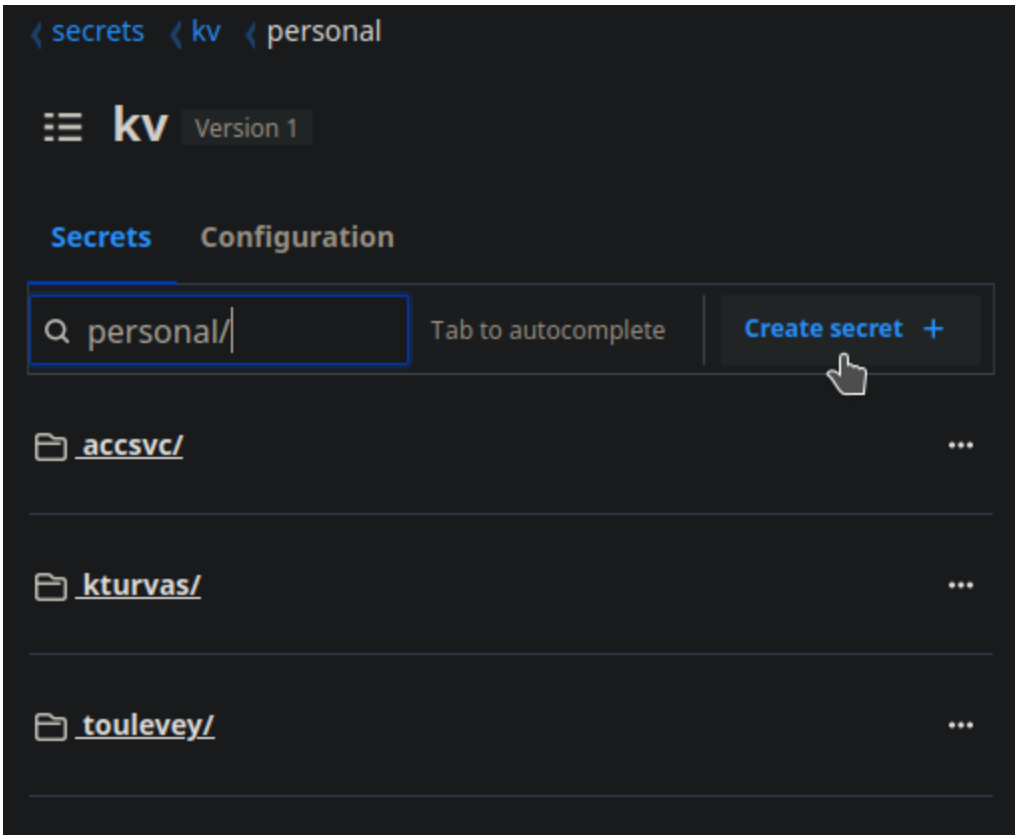
Personal secrets

Personal secrets path is a path that only one account has access to, the path where you create secrets that are specific to your use cases.

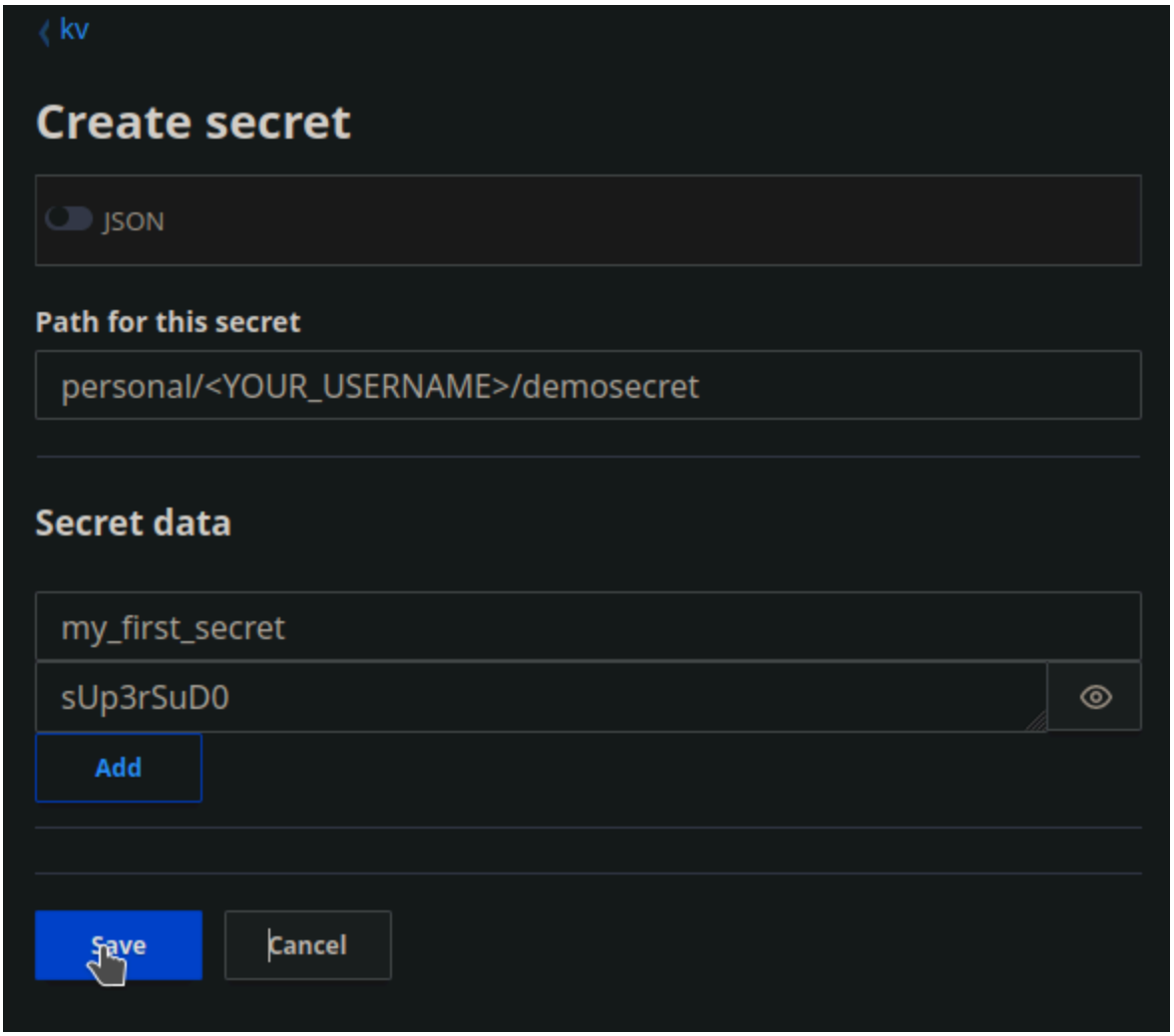
Click on the `personal/` path:



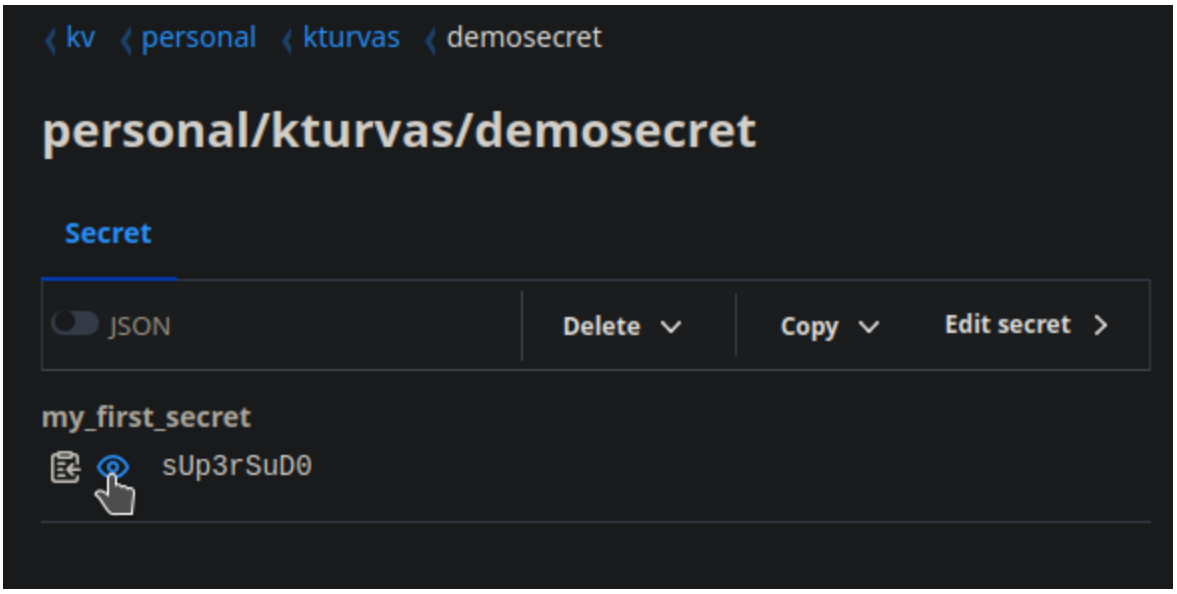
Click on "Create secret":



Set key/value secrets under your personal user secrets path:



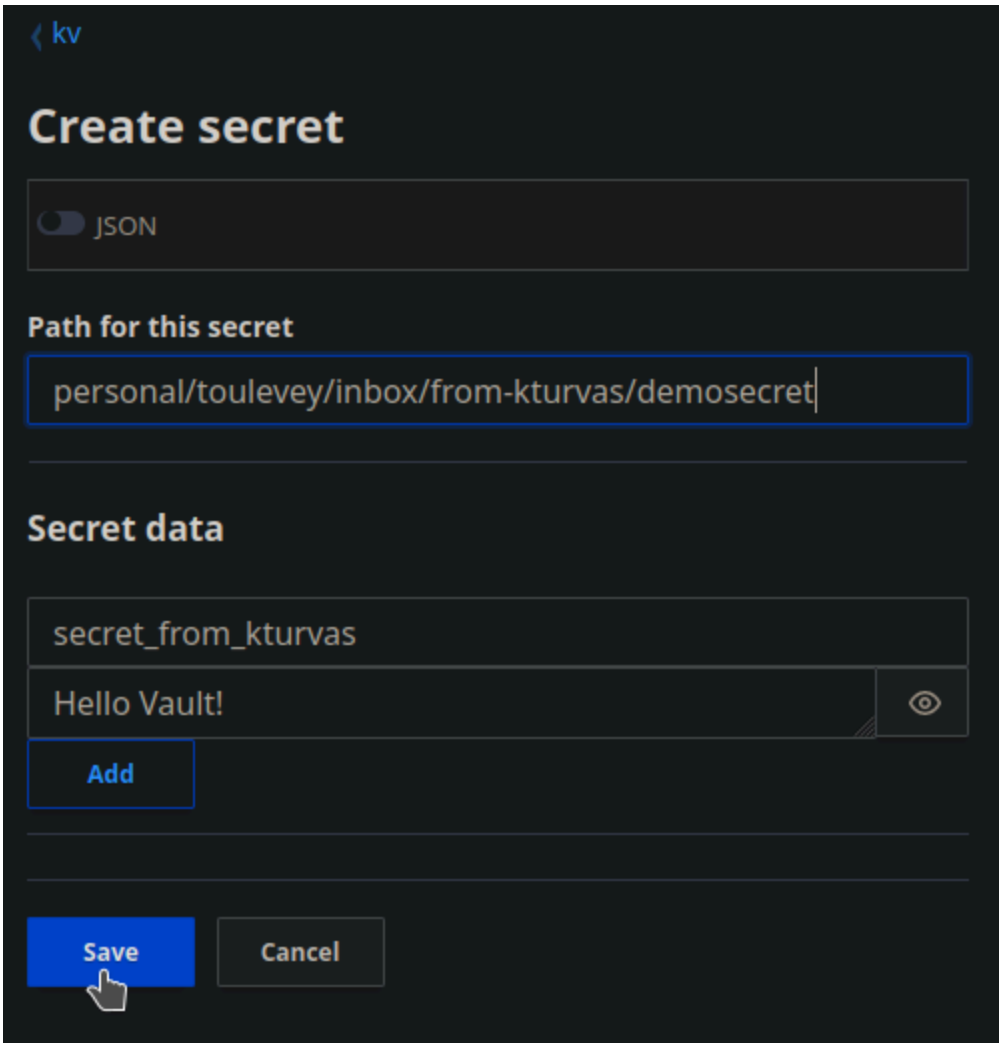
Now you should be able to see the new secret you created:



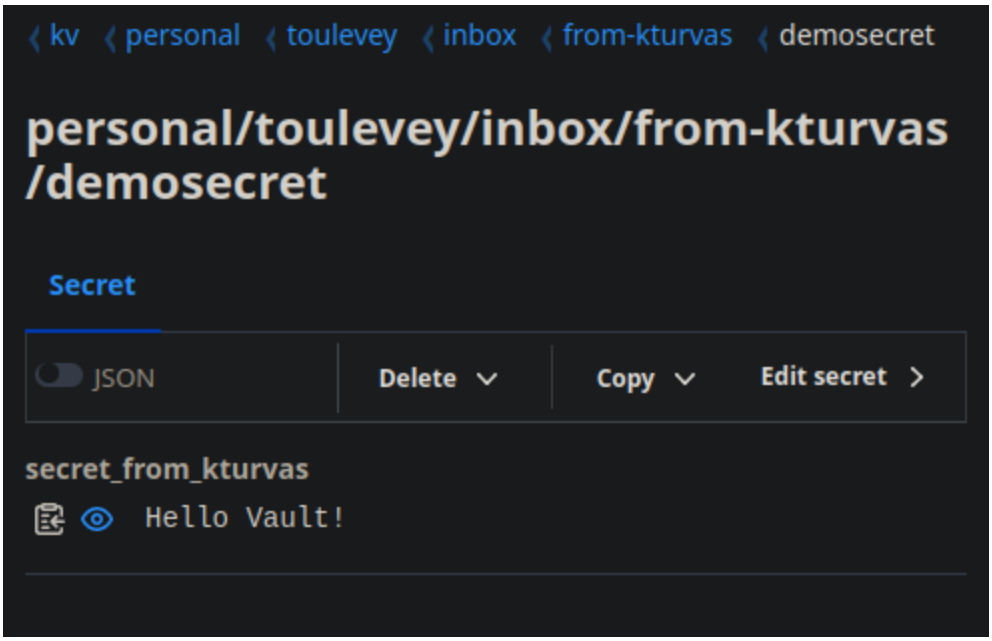
Inbox secrets

Inbox secrets is a path template that allows sharing secrets with one account. Under every personal secret path there is a special sub path called inbox where other users have read/write access to (this is the only path in the personal secrets path where others have read/write access to). That path is `personal/<USERNAME>/inbox/from-<ANY OTHER USERNAME>/*`

Create a new secret under someone elses inbox:



Note that you only have access to other people's personal secrets path if it is under the inbox paths `kv/personal/*/from-"$USER"` and `kv/personal/*/"$USER"`:



General secrets

General secrets correspond to some policy that allows reading/writing secrets shared by a group of users (like users of an e-group/LDAP group). The following requires that you are part of the `acc-adm-support` e-group for example.

Using what you've learned in the previous steps, try creating a secret under the path `kv/acc-adm` and then try fetching it or use the web UI to see what secrets others have created under `kv/acc-adm`.